

# Transition Invariants Revisited: Termination Witnesses and Their Validation

---

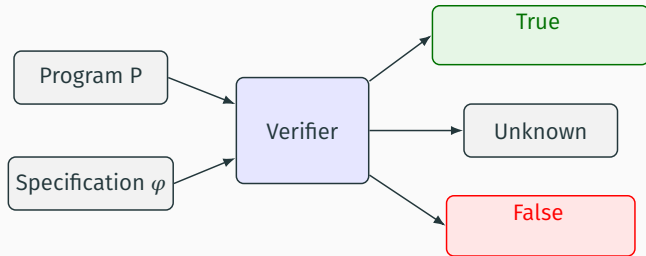
Dirk Beyer , **Marek Jankola** , Marian Lingsch-Rosenfeld 



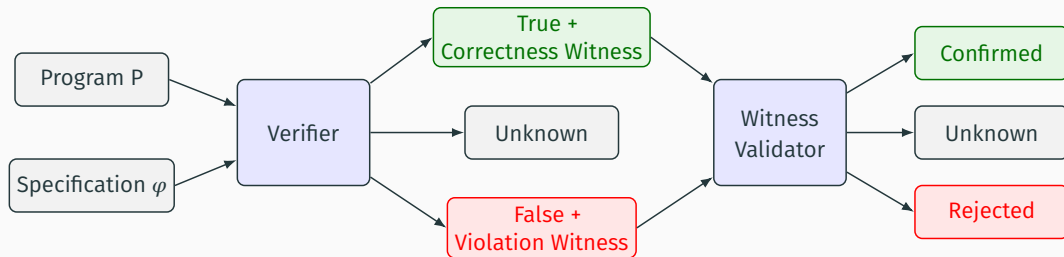
September 23, 2026  
AVM 2026, Eger, Hungary



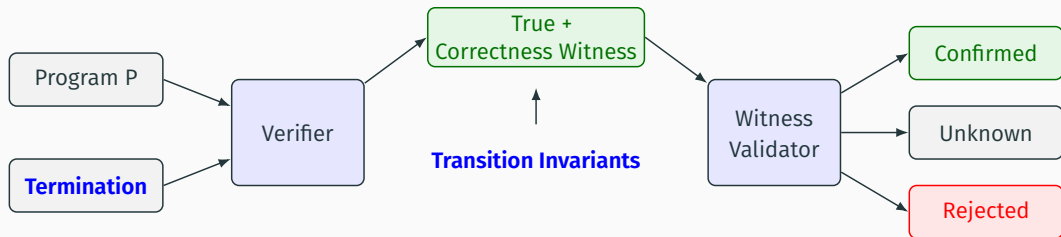
# Motivation



# Motivation



# Motivation



A loop is terminating



there exists a **disjunctively** well-founded transition invariant [12]

# Overview

Background

Format

Construction of Transition Invariants

Validation of Termination Witnesses

Evaluation

Conclusion

# Overview

Background

Format

Construction of Transition Invariants

Validation of Termination Witnesses

Evaluation

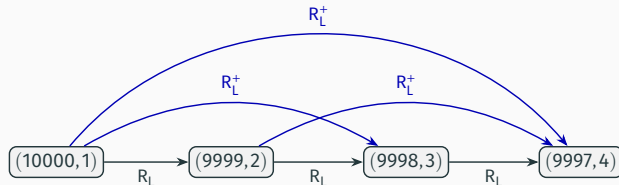
Conclusion

# Transition Relation $R_L$ of a Loop

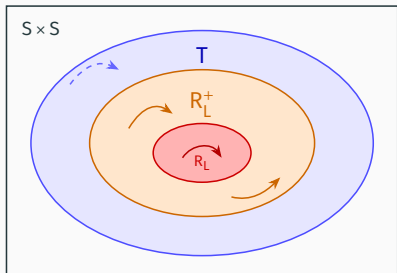
```
1 int main(void) {  
2   int i, j;  
3   j = 1;  
4   i = 10000;  
5   while (i - j >= 1) {  
6     j = j + 1;  
7     i = i - 1;  
8   }  
9   return 0;  
10 }
```

$$R_L(s, s') \equiv \underbrace{s(i) - s(j) \geq 1}_{\text{loop condition}} \wedge s'(i) = s(i) - 1 \wedge s'(j) = s(j) + 1$$

$$R_L^+(s, s') \equiv \exists k \geq 1 : s(i) - s(j) \geq 2k - 1 \wedge s'(i) = s(i) - k \wedge s'(j) = s(j) + k$$

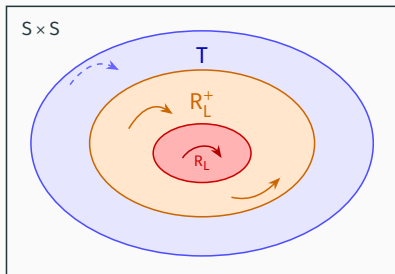


# Disjunctively Well-Founded Transition Invariants

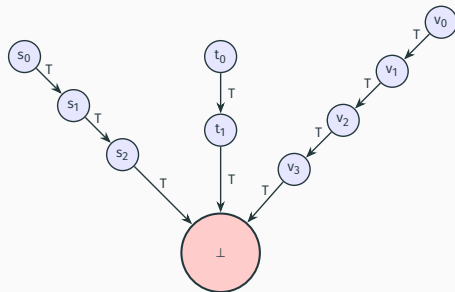


$T$  over-approximates all paths in  $R_L^+$ :  $R_L^+ \subseteq T$

# Disjunctively Well-Founded Transition Invariants

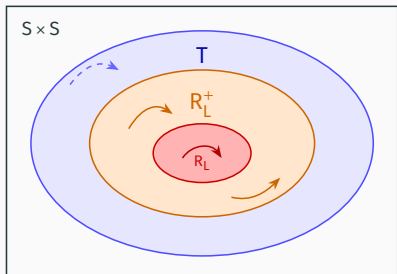


$T$  over-approximates all paths in  $R_L^+$ :  $R_L^+ \subseteq T$

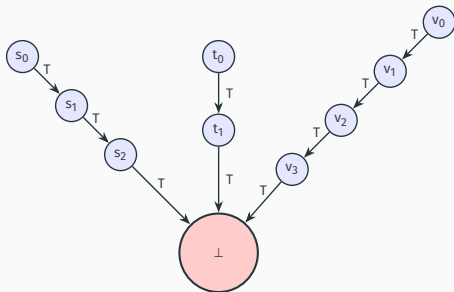


every  $T$ -path is finite: no infinite  $s_0, s_1, s_2, \dots$

# Disjunctively Well-Founded Transition Invariants



$T$  over-approximates all paths in  $R_L^+$ :  $R_L^+ \subseteq T$



every  $T$ -path is finite: no infinite  $s_0, s_1, s_2, \dots$

The relation  $T$  is **disjunctively** well-founded if it can be divided into  $T = T_1 \cup \dots \cup T_n$ , where  $T_1, \dots, T_n$  are well-founded.

# Overview

Background

Format

Construction of Transition Invariants

Validation of Termination Witnesses

Evaluation

Conclusion

# Format

```
1 int main(void) {  
2     int i, j;  
3     j = 1;  
4     i = 10000;  
5     while (i-j >= 1) {  
6         j = j + 1;  
7         i = i - 1;  
8     }  
9     return 0;  
10 }
```

- entry\_type: invariant\_set  
metadata: ...  
content:
  - invariant:
    - type: loop\_invariant
    - location:
      - line: 5
    - value: "i >= j"
    - format: c\_expression
  - invariant:
    - type: loop\_transition\_invariant
    - location:
      - line: 5
    - value: "\\at(i, AnyPrev) - \\at(j, AnyPrev) > i - j"
    - format: ext\_c\_expression

Based on software verification witnesses v2.0 [1]

# Overview

Background

Format

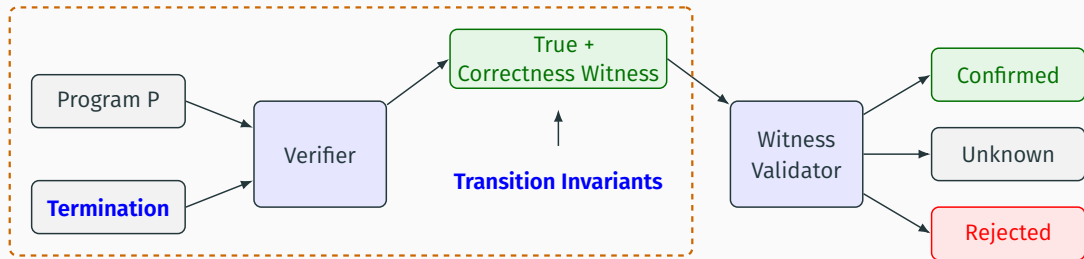
Construction of Transition Invariants

Validation of Termination Witnesses

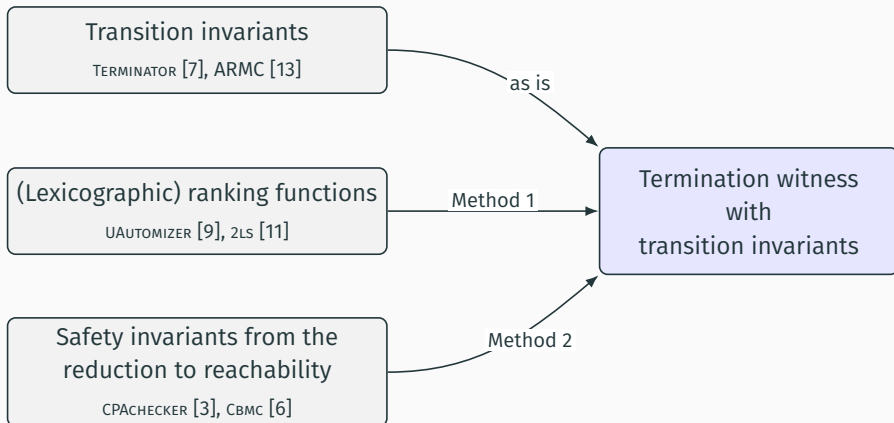
Evaluation

Conclusion

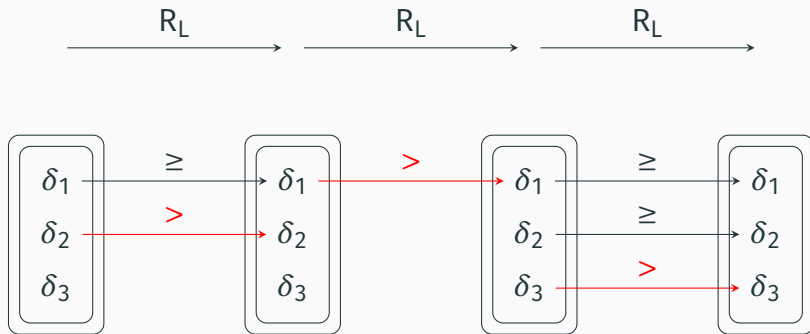
# Construction of Transition Invariants



# Why Transition Invariants?



# Method 1: Lexicographic Ranking Functions



$$\bigvee_{1 \leq i \leq k} \delta_i(s) > \delta_i(s') \wedge \bigwedge_{1 \leq j < i} (\delta_j(s) \geq \delta_j(s'))$$

# Method 1: From Ranking Functions to Transition Invariants

## Ranking function

```
1 int main(void) {  
2     int i, j;  
3     j = 1;  
4     i = 10000;  
5     while (i - j >= 1) {  
6         j = j + 1;  
7         i = i - 1;  
8     }  
9     return 0;  
10 }
```

$$\rho(s) = s(i) - s(j)$$

## Lexicographic ranking function

```
1 int main(void) {  
2     int x, y;  
3     x = nondet();  
4     y = nondet();  
5     while (x > 0 && y > 0) {  
6         if (nondet()) {  
7             x = x - 1;  
8             y = nondet();  
9         } else  
10            y = y - 1;  
11     }  
12 }
```

$$\langle \rho_0, \rho_1 \rangle = \langle s(x), s(y) \rangle$$

# Method 1: From Ranking Functions to Transition Invariants

## Ranking function

```
1 int main(void) {  
2     int i, j;  
3     j = 1;  
4     i = 10000;  
5     while (i - j >= 1) {  
6         j = j + 1;  
7         i = i - 1;  
8     }  
9     return 0;  
10 }
```

## Lexicographic ranking function

```
1 int main(void) {  
2     int x, y;  
3     x = nondet();  
4     y = nondet();  
5     while (x > 0 && y > 0) {  
6         if (nondet()) {  
7             x = x - 1;  
8             y = nondet();  
9         } else  
10            y = y - 1;  
11     }  
12 }
```

$$\rho(s) = s(i) - s(j)$$

$$\langle \rho_0, \rho_1 \rangle = \langle s(x), s(y) \rangle$$

$$T(s, s') \equiv s(i) - s(j) > s'(i) - s'(j)$$

# Method 1: From Ranking Functions to Transition Invariants

## Ranking function

```
1 int main(void) {  
2     int i, j;  
3     j = 1;  
4     i = 10000;  
5     while (i - j >= 1) {  
6         j = j + 1;  
7         i = i - 1;  
8     }  
9     return 0;  
10 }
```

$$\rho(s) = s(i) - s(j)$$

$$T(s, s') \equiv s(i) - s(j) > s'(i) - s'(j)$$

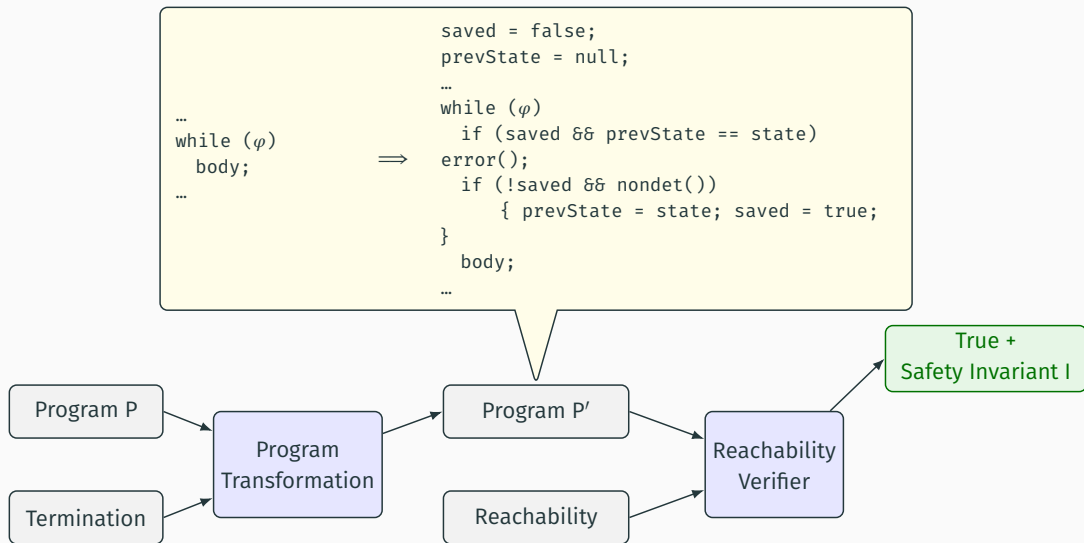
## Lexicographic ranking function

```
1 int main(void) {  
2     int x, y;  
3     x = nondet();  
4     y = nondet();  
5     while (x > 0 && y > 0) {  
6         if (nondet()) {  
7             x = x - 1;  
8             y = nondet();  
9         } else  
10            y = y - 1;  
11     }  
12 }
```

$$\langle \rho_0, \rho_1 \rangle = \langle s(x), s(y) \rangle$$

$$T(s, s') \equiv s(x) > s'(x) \vee \\ (s(x) \geq s'(x) \wedge s(y) > s'(y))$$

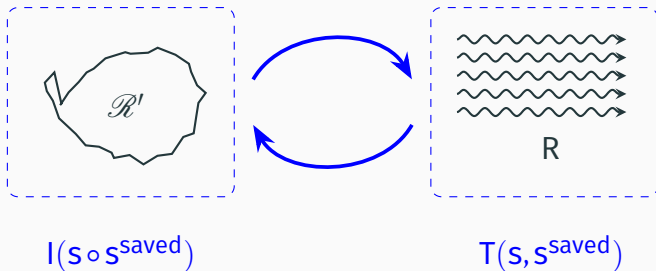
## Method 2: Reduction Approaches



## Method 2: Theorem for Reduction Approaches

Safe 1-step Invariant

Well-founded transition invariant



## Method 2: From Safe Invariants to Transition Invariants

```
1  int main(void) {
2      int saved = 0;
3      int _i, _j;
4      int i, j;
5      j = 1;
6      i = 10000;
7      while (i-j >= 1) {
8          if (saved && _i == i && _j == j)
9              error();
10         if (!saved && nondet()) {
11             _i = i; _j = j; saved = 1;
12         }
13         j = j + 1;
14         i = i - 1;
15     }
16     return 0;
17 }
```

A reachability verifier proves  $P'$  safe — no state is revisited — and reports the **safe 1-step invariant** for the loop head:

$$I \equiv \text{saved} = 0 \vee \_i - \_j > i - j$$

## Method 2: From Safe Invariants to Transition Invariants

```
1  int main(void) {
2      int saved = 0;
3      int _i, _j;
4      int i, j;
5      j = 1;
6      i = 10000;
7      while (i-j >= 1) {
8          if (saved && _i == i && _j == j)
9              error();
10         if (!saved && nondet()) {
11             _i = i; _j = j; saved = 1;
12         }
13         j = j + 1;
14         i = i - 1;
15     }
16     return 0;
17 }
```

A reachability verifier proves  $P'$  safe — no state is revisited — and reports the **safe 1-step invariant** for the loop head:

$$I \equiv \text{saved} = 0 \vee \_i - \_j > i - j$$

Instantiating  $\text{saved} = 1$  yields the **transition invariant**:

$$T(s, s') \equiv s(i) - s(j) > s'(i) - s'(j)$$

# Overview

Background

Format

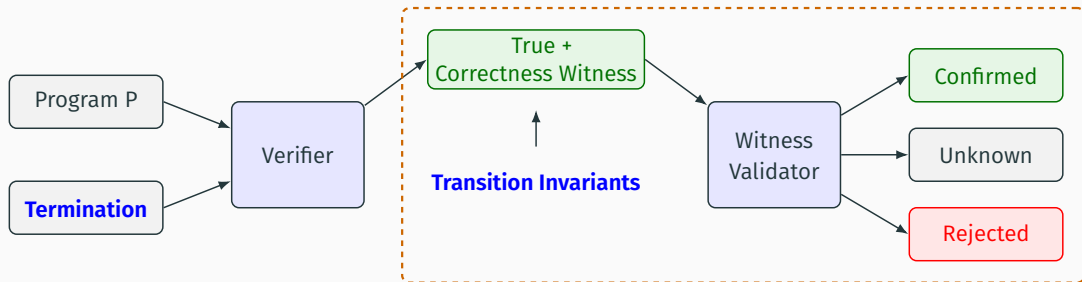
Construction of Transition Invariants

**Validation of Termination Witnesses**

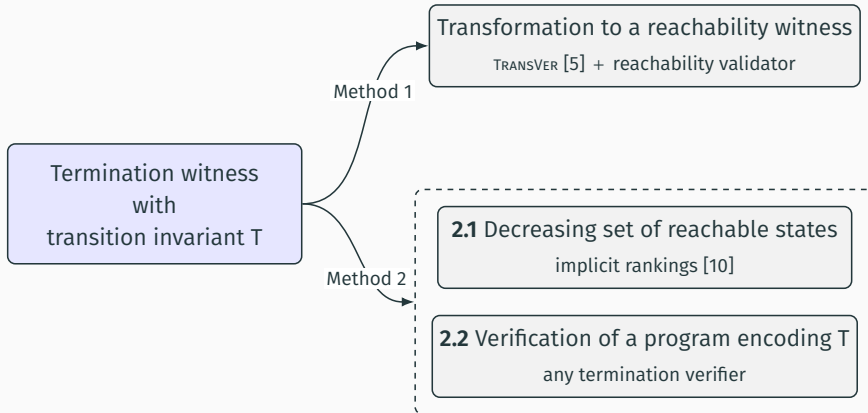
Evaluation

Conclusion

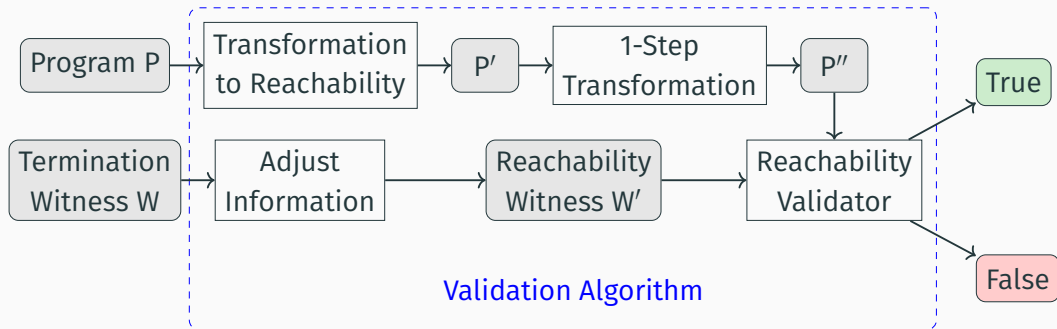
# Validation of Termination Witnesses



# How to Validate a Termination Witness?



# Method 1: Validation via Transformation



## Method 2: General Validation Framework

Given a candidate transition invariant  $T$ , we have to

1. see that it is a transition invariant

## Method 2: General Validation Framework

Given a candidate transition invariant  $T$ , we have to

1. see that it is a transition invariant

**Base Case:**  $\forall 1 \leq i \leq k : \forall s_0, \dots, s_i : \left( \bigwedge_{0 \leq j \leq i} I_L(s_j) \bigwedge_{0 \leq j < i} R_L(s_j, s_{j+1}) \right) \Rightarrow T(s_0, s_i)$

**Step Case:**  $\forall s, s_0, \dots, s_k : \left( I_L(s) \wedge T(s, s_0) \bigwedge_{0 \leq j \leq k} I_L(s_j) \bigwedge_{0 \leq j < k} R_L(s_j, s_{j+1}) \right) \Rightarrow T(s, s_k)$

## Method 2: General Validation Framework

Given a candidate transition invariant  $T$ , we have to

1. see that it is a transition invariant

**Base Case:**  $\forall 1 \leq i \leq k : \forall s_0, \dots, s_i : \left( \bigwedge_{0 \leq j \leq i} I_L(s_j) \bigwedge_{0 \leq j < i} R_L(s_j, s_{j+1}) \right) \Rightarrow T(s_0, s_i)$

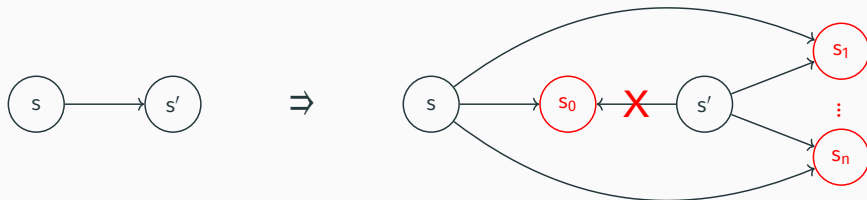
**Step Case:**  $\forall s, s_0, \dots, s_k : \left( I_L(s) \wedge T(s, s_0) \bigwedge_{0 \leq j \leq k} I_L(s_j) \bigwedge_{0 \leq j < k} R_L(s_j, s_{j+1}) \right) \Rightarrow T(s, s_k)$

2. see if it is well-founded: two approaches

## Method 2.1: Validation with Decreasing set of reachable states

**Idea:** Show that there exists a ranking function for  $T$ , without actually constructing it<sup>1</sup>.

**Decreasing set of reachable states:**



<sup>1</sup>Lotan, R., Shoham, S.: Implicit rankings for verifying liveness properties in first-order logic. In: Proc. TACAS. pp. 375–395. Springer (2025) [10]

## Method 2.2: Validation with Verification

**Idea:** Encode  $T$  as a program and prove that it is terminating with any software verifier.

```
1  int main() {  
2    int x = nondet();  
3    int y = nondet();  
4    int xPrev = nondet();  
5    int yPrev = nondet();  
6  
7    while (( $T(xPrev, yPrev, x, y)$ )) {  
8      xPrev = x;  
9      yPrev = y;  
10     x = nondet();  
11     y = nondet();  
12   }  
13 }
```

## Method 2.2: Example

**Running example:**  $T(s, s') \equiv s(i) - s(j) > s'(i) - s'(j)$

```
1  int main() {  
2    int i = nondet();  
3    int j = nondet();  
4    int iPrev = nondet();  
5    int jPrev = nondet();  
6  
7    while ((iPrev - jPrev > i - j)) {  
8      iPrev = i;  
9      jPrev = j;  
10     i = nondet();  
11     j = nondet();  
12   }  
13 }
```

# Overview

Background

Format

Construction of Transition Invariants

Validation of Termination Witnesses

Evaluation

Conclusion

# Experimental Setting

| Verifier                                                                        | Termination argument                     | Method |
|---------------------------------------------------------------------------------|------------------------------------------|--------|
| CPACHECKER [2]-TRANSVER [5]<br>UAUTOMIZER [8]-TRANSVER<br>GOBLINT [14]-TRANSVER | safety invariant of the reduction        | 2      |
| CPACHECKER                                                                      | lexicographic ranking function           | 1      |
| Validator                                                                       | Validation approach                      | Method |
| CPACHECKER-TRANSVER<br>UAUTOMIZER-TRANSVER                                      | transformation to a reachability witness | 1      |
| CPACHECKER-decreasing                                                           | decreasing set of reachable states       | 2.1    |
| CPACHECKER-simplifying                                                          | verification of a program encoding T     | 2.2    |

# Results

| Verifier               | Witnesses | Validated  | Validator              | Validated                   |
|------------------------|-----------|------------|------------------------|-----------------------------|
| UAUTOMIZER<br>TRANSVER | 389       | 375 (96 %) | CPACHECKER-TRANSVER    | 262 (107) <sup>†</sup>      |
|                        |           |            | UAUTOMIZER-TRANSVER    | <b>350</b> -                |
|                        |           |            | CPACHECKER-decreasing  | 205 (28) <sup>†</sup>       |
|                        |           |            | CPACHECKER-simplifying | 147 (10) <sup>†</sup>       |
| GOBLINT<br>TRANSVER    | 286       | 176 (62 %) | CPACHECKER-TRANSVER    | <b>155</b> (3) <sup>†</sup> |
|                        |           |            | UAUTOMIZER-TRANSVER    | 128 (5) <sup>†</sup>        |
|                        |           |            | CPACHECKER-decreasing  | 85 (0)                      |
|                        |           |            | CPACHECKER-simplifying | 85 (0)                      |
| CPACHECKER             | 422       | 366 (87 %) | CPACHECKER-TRANSVER    | 131 (32) <sup>†</sup>       |
|                        |           |            | UAUTOMIZER-TRANSVER    | 253 (0)                     |
|                        |           |            | CPACHECKER-decreasing  | <b>273</b> -                |
|                        |           |            | CPACHECKER-simplifying | 261 -                       |

In **green**: tasks validated by the validator but not verified by its corresponding verifier

# Overview

Background

Format

Construction of Transition Invariants

Validation of Termination Witnesses

Evaluation

Conclusion

# Conclusion

We presented:

- a termination witness format based on transition invariants,
- ways to express popular termination arguments as transition invariants,
- three orthogonal validation techniques,
- and evaluated them in multiple state-of-the-art tools.



Paper [4]

## References i

- [1] Ayaziová, P., Beyer, D., Lingsch-Rosenfeld, M., Spiessl, M., Strejček, J.: Software verification witnesses 2.0. In: Proc. SPIN. pp. 184–203. LNCS 14624, Springer (2024).  
[https://doi.org/10.1007/978-3-031-66149-5\\_11](https://doi.org/10.1007/978-3-031-66149-5_11)
- [2] Baier, D., Beyer, D., Chien, P.C., Jakobs, M.C., Jankola, M., Kettl, M., Lee, N.Z., Lemberger, T., Lingsch-Rosenfeld, M., Wachowitz, H., Wendler, P.: Software verification with CPACHECKER 3.0: Tutorial and user guide (extended version). arXiv/CoRR (2024)

## References ii

- [3] Baier, D., Beyer, D., Chien, P.C., Jankola, M., Kettl, M., Lee, N.Z., Lemberger, T., Lingsch-Rosenfeld, M., Spiessl, M., Wachowitz, H., Wendler, P.: CPACHECKER 2.3 with strategy selection (competition contribution). In: Proc. TACAS (3). pp. 359–364. LNCS 14572, Springer (2024). [https://doi.org/10.1007/978-3-031-57256-2\\_21](https://doi.org/10.1007/978-3-031-57256-2_21)
- [4] Beyer, D., Jankola, M., Lingsch-Rosenfeld, M.: Transition invariants revisited: Termination witnesses and their validation. In: Proc. CAV. pp. 28–53. LNCS, Springer (2026). [https://doi.org/10.1007/978-3-032-32537-2\\_2](https://doi.org/10.1007/978-3-032-32537-2_2)

## References iii

- [5] Beyer, D., Jankola, M., Lingsch-Rosenfeld, M., Xia, T., Zheng, X.: TRANSVER: A modular program-transformation framework for reduction to reachability. In: Proc. SPIN. pp. 1–24. LNCS 15945, Springer (2025). [https://doi.org/10.1007/978-3-032-06847-7\\_1](https://doi.org/10.1007/978-3-032-06847-7_1)
- [6] Clarke, E.M., Kröning, D., Lerda, F.: A tool for checking ANSI-C programs. In: Proc. TACAS. pp. 168–176. LNCS 2988, Springer (2004). [https://doi.org/10.1007/978-3-540-24730-2\\_15](https://doi.org/10.1007/978-3-540-24730-2_15)
- [7] Cook, B., Podelski, A., Rybalchenko, A.: TERMINATOR: Beyond safety. In: Proc. CAV. pp. 415–418. LNCS 4144, Springer (2006). [https://doi.org/10.1007/11817963\\_37](https://doi.org/10.1007/11817963_37)

## References iv

- [8] Heizmann, M., Barth, M., Dietsch, D., Fichtner, L., Hoenicke, J., Klumpp, D., Naouar, M., Schindler, T., Schüssele, F., Podelski, A.: ULTIMATE AUTOMIZER 2023 (competition contribution). In: Proc. TACAS (2). pp. 577–581. LNCS 13994, Springer (2023).  
[https://doi.org/10.1007/978-3-031-30820-8\\_39](https://doi.org/10.1007/978-3-031-30820-8_39)
- [9] Heizmann, M., Hoenicke, J., Leike, J., Podelski, A.: Linear ranking for linear lasso programs. In: Proc. ATVA. pp. 365–380. LNCS 8172, Springer (2013). [https://doi.org/10.1007/978-3-319-02444-8\\_26](https://doi.org/10.1007/978-3-319-02444-8_26)
- [10] Lotan, R., Shoham, S.: Implicit rankings for verifying liveness properties in first-order logic. In: Proc. TACAS. pp. 375–395. Springer (2025). [https://doi.org/10.1007/978-3-031-90643-5\\_20](https://doi.org/10.1007/978-3-031-90643-5_20)

## References v

- [11] Malík, V., Schrammel, P., Vojnar, T., Nečas, F.: 2LS: Arrays and loop unwinding (competition contribution). In: Proc. TACAS (2). pp. 529–534. LNCS 13994, Springer (2023).  
[https://doi.org/10.1007/978-3-031-30820-8\\_31](https://doi.org/10.1007/978-3-031-30820-8_31)
- [12] Podelski, A., Rybalchenko, A.: Transition invariants. In: Proc. LICS. pp. 32–41. IEEE (2004).  
<https://doi.org/10.1109/LICS.2004.1319598>
- [13] Podelski, A., Rybalchenko, A.: Transition predicate abstraction and fair termination. In: Proc. POPL. pp. 132–144. ACM (2005).  
<https://doi.org/10.1145/1040305.1040317>

- [14] Saan, S., Schwarz, M., Erhard, J., Pietsch, M., Seidl, H., Tilscher, S., Vojdani, V.: GOBLINT: Autotuning thread-modular abstract interpretation (competition contribution). In: Proc. TACAS (2). pp. 547–552. LNCS 13994, Springer (2023).  
[https://doi.org/10.1007/978-3-031-30820-8\\_34](https://doi.org/10.1007/978-3-031-30820-8_34)